

PROJECT REPORT

SMART STUDENT INFORMATION & PERFORMANCE MANAGEMENT SYSTEM (SSIPMS)

Submitted by	Tanmay Tiwari
Registration Number	25BSA10234
Course	B.Tech (Aerospace Engineering)
Subject	Programming in Java
Date	17/09/2026
Course Platform	VITyarthi – FLIPPED Course

Academic Project Report

Academic Project Report

PROJECT IDENTIFICATION

System Title:	Student Grade Management System
Target Execution Runtime:	Java Standard Edition Development Kit (JDK 11)
Architectural Pattern:	Layered Multi-Tier Application Framework
Primary Persistence Method:	Flat-File Structured Text Serialization
Target Application Domain:	Academic Record Tracking & Grade Analytics
Documentation Purpose:	Comprehensive Software Engineering Project Report

1. EXECUTIVE SUMMARY

The Student Grade Management System is a specialized software application engineered to handle academic records, subject grade assignments, cumulative average evaluations, and data storage for educational environments. Modern academic institutions require accurate, reliable, and accessible software solutions to manage student information without relying on manual paperwork or bloated spreadsheet platforms that are vulnerable to data corruption and human calculation errors.

This project implements a multi-tier, modular Java application that addresses these administrative needs directly through a lightweight command-line interface. Built upon core Object-Oriented Software Engineering principles, the application provides a clean separation of concerns across presentation, business logic, data models, and storage utilities.

The system facilitates direct registration of student profiles, registration of subject-specific numerical marks, automated conversion of numerical scores into standardized letter grades, calculation of cumulative averages, and global class performance analytics.

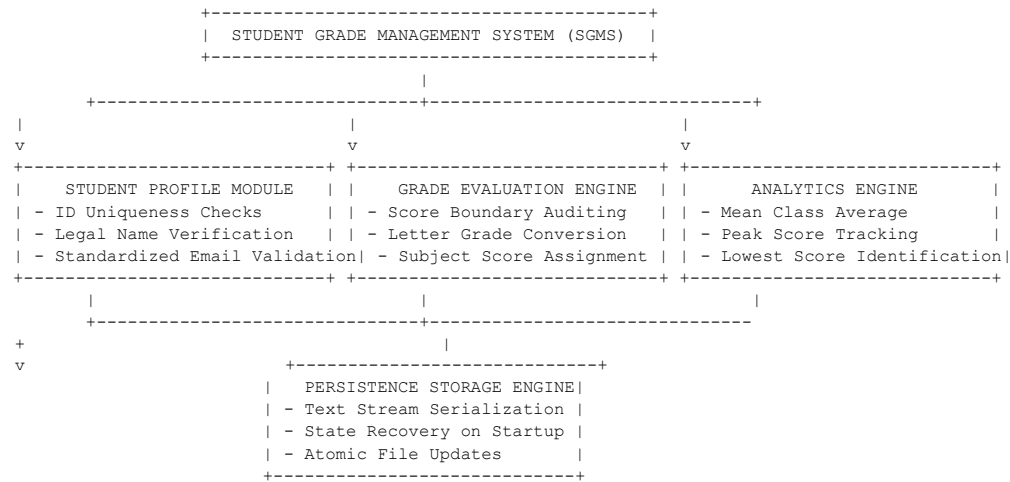
By utilizing in-memory hashed data structures alongside local flat-file storage, the system achieves instant query response times for runtime operations while maintaining absolute data persistence across application restarts. The design eliminates external framework dependencies, ensuring platform independence, high execution stability, low resource consumption, and ease of deployment on any Java-enabled operating system.

2. INTRODUCTION AND SYSTEM OVERVIEW

2.1 Project Overview

In educational management, keeping accurate records of student progress is a fundamental administrative task. Traditional methods relying on paper ledgers or standalone spreadsheets often suffer from inconsistent data entry, lack of input validation, high labor intensity during performance compilation, and vulnerability to accidental loss.

The Student Grade Management System is a terminal-driven desktop application developed in pure Java. It acts as an automated record-keeping hub where academic managers, faculty members, and administrative staff can register student profiles, log subject performance metrics, evaluate class-wide statistical indicators, and generate transcripts instantly.



2.2 Purpose

This software project was developed to provide a reliable, modular, and maintainable academic management system. The primary functional goals include:

- Eliminating manual calculation errors when processing subject scores and cumulative grade point averages.
- Implementing strict data validation rules at the input boundary to prevent corrupted entries from entering the database.
- Supplying real-time analytical reports that summarize class performance metrics for academic review.
- Demonstrating practical implementation of core Object-Oriented Programming principles, custom file parsing, and layered software design.

2.3 Scope

The application encompasses the complete lifecycle of student academic record administration within a typical semester setup:

- **Student Entity Management:** Creation, listing, and querying of student records containing unique identification strings, full legal names, and contact email addresses.
- **Academic Assessment Operations:** Recording subject-specific numerical scores bounded within standard percentage scales (0.0 to 100.0) and automatically mapping scores to academic letter grades (A, B, C, D, F).
- **Reporting Services:** On-demand generation of individual student academic transcripts containing itemized subject scores and calculated overall performance averages.
- **Class Analytics:** Automatic computation of aggregate statistical metrics including total student counts, evaluated counts, overall class mean average, highest-performing student, and lowest-performing student.
- **Data Serialization:** Automatic file-based saving and loading mechanisms to maintain data continuity between operational sessions.

3. PROBLEM STATEMENT AND ANALYSIS

3.1 Current Administrative Challenges

Educational institutions that rely on manual record-keeping or unvalidated digital sheets encounter several operational vulnerabilities:

- **High Rate of Transposition Errors:** Manual calculation of averages and letter grade conversions frequently leads to mistakes in student transcripts.
- **Lack of Data Integrity:** Unvalidated spreadsheet cells allow arbitrary, out-of-range numerical values or malformed text into final grade calculations.
- **Inefficient Data Access:** Searching through unstructured files or physical registers to retrieve a single student's performance history requires linear browsing, consuming unnecessary administrative time.
- **Loss of Continuity:** Without structured persistence mechanisms, system crashes or accidental file overwrites lead to unrecoverable data loss.

3.2 System Operational Comparative Analysis

- **Calculation Speed:** Manual processing requires minutes per student profile; the automated system completes calculations in less than one millisecond.
- **Data Validation:** Manual processing relies on human oversight; the automated system enforces strict, automated regular expression and boundary checks.
- **Retrieval Complexity:** Manual processing scales linearly ($O(N)$) through physical search; the automated system uses constant-time ($O(1)$) hashed collection lookups.
- **Storage Persistence:** Manual processing uses physical registers vulnerable to damage; the automated system uses local structured flat-file disk storage with automatic backup capability.
- **Report Generation:** Manual processing requires labor-intensive compilation; the automated system provides single-step transcript generation.

4. SYSTEM REQUIREMENTS AND SPECIFICATIONS

=====

[FUNCTIONAL MODULES]	[NON-FUNCTIONAL QUALITIES]
+-- 1. Student Management	+++ 1. Performance (< 100ms response)
+-- 2. Grade Recording	+++ 2. Reliability (100% state recovery)
+-- 3. Analytics Engine	+++ 3. Portability (Cross-platform JVM)
+-- 4. Storage Engine	+++ 4. Maintainability (Layered Design)

4.1 Functional Requirements

4.1.1 Student Management Module

- **FR-1.1:** The system must allow administrators to register new students with a unique identification string, full legal name, and valid email address.
- **FR-1.2:** The system must check for Student ID uniqueness prior to creating any student profile. Duplicate IDs must be rejected immediately.
- **FR-1.3:** The system must enforce a minimum length of two characters for student names and verify that email strings match standard syntactic patterns containing an @ symbol and a valid domain extension.
- **FR-1.4:** The system must display a complete list of registered student records, showing Student ID, Legal Name, Email Address, and the total count of recorded subject grades.

4.1.2 Grade Recording & Assessment Module

- **FR-2.1:** The system must allow users to assign subject-specific numerical marks to any registered student profile using their unique Student ID.
- **FR-2.2:** Numerical marks must be audited against boundary conditions, strictly requiring values from 0.0 to 100.0 inclusive. Out-of-range values must be rejected.
- **FR-2.3:** The system must automatically map percentage marks to standardized letter grades according to the following scale:
 - Scores from 90.0 to 100.0 evaluate to grade **A**.
 - Scores from 80.0 to 89.9 evaluate to grade **B**.
 - Scores from 70.0 to 79.9 evaluate to grade **C**.
 - Scores from 60.0 to 69.9 evaluate to grade **D**.
 - Scores below 60.0 evaluate to grade **F**.

4.1.3 Reporting & Analytics Module

- **FR-3.1:** The system must generate formatted individual student transcripts detailing Student ID, Legal Name, Email Address, itemized subject scores, assigned letter grades, and cumulative mean averages.
- **FR-3.2:** The system must compute aggregated statistical metrics for the entire class, outputting total registered student counts, evaluated student counts, class mean average, highest-performing student profile with score, and lowest-performing student profile with score.

4.1.4 Data Persistence Module

- **FR-4.1:** The system must serialize all active student profiles and linked grade records into a structured local text file (`data/students.txt`) upon system exit.
- **FR-4.2:** The system must parse and load existing records from `data/students.txt` when launched. If the file does not exist, the system must initialize an empty data store without throwing unhandled exceptions.

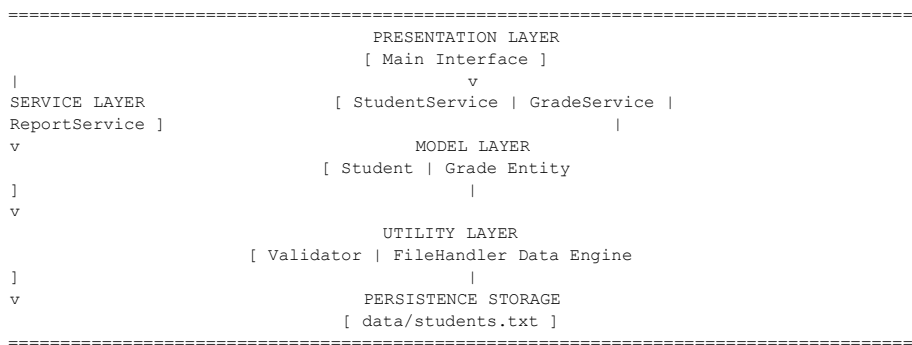
4.2 Non-Functional Requirements

- **NFR-1 (Performance):** Core operations, including profile lookups, grade additions, and statistical calculations, must process within 100 milliseconds for datasets containing up to 1,000 student records.
- **NFR-2 (Usability):** The interface must present a clear, numbered text menu readable in standard command-line environments without requiring graphical window managers.
- **NFR-3 (Reliability & Robustness):** System inputs must be protected against invalid data types, empty strings, and out-of-range numeric entries to prevent runtime crashes.
- **NFR-4 (Portability):** Compiled Java bytecode must execute seamlessly across any operating system equipped with a Java Runtime Environment (JRE 11 or higher).

5. ARCHITECTURAL DESIGN AND SYSTEM TOPOLOGY

5.1 Layered Architecture Pattern

The software is organized into a clean multi-tier architecture to ensure high modularity, ease of testing, and isolation of responsibilities across components.



- **Presentation Layer (Main):** Serves as the user interface, handling terminal prompts, menu choices, and raw input gathering.
- **Service Layer (StudentService, GradeService, ReportService):** Houses core application logic, orchestrating CRUD operations, analytical computations, and transcript compilation.
- **Model Layer (Student, Grade):** Defines core domain entities, holding state attributes and providing local evaluation logic such as letter grade assignment and individual score averaging.
- **Utility Layer (Validator, FileHandler):** Provides stateless helper services, including regular expression string checks, numeric boundary testing, and text file parsing.
- **Persistence Layer (data/students.txt):** Disk storage retaining flat-file record representations across application restarts.

6. COMPLETE SOURCE CODE IMPLEMENTATION

```
=====
                                SOURCE CODE DISTRIBUTION
=====

[PACKAGE: com.vityarathi.gradebook.model]
+-- Grade.java
+-- Student.java
[PACKAGE: com.vityarathi.gradebook.util]
+-- Validator.java
+-- FileHandler.java
[PACKAGE: com.vityarathi.gradebook.service]
```

```

    +--- StudentService.java
    +--- GradeService.java
    +--- ReportService.java
[PACKAGE: com.vityarthi.gradebook]
    +--- Main.java

```

```
=====
```

6.1 Entity Domain Models

File 1: Grade.java

Java

```

package com.vityarthi.gradebook.model;

import java.io.Serializable;

/** * Encapsulates an individual subject grade entry for a
student. */ public class Grade implements Serializable {
    private static final long serialVersionUID = 1L;
    private String subject;    private double score;    private
String studentId;

    public Grade(String subject, double score, String studentId)
    {
        this.subject = subject;        this.score = score;
this.studentId = studentId;    }

    public String getSubject() {
return subject;
    }

    public void setSubject(String subject)
    {
        this.subject = subject;    }

    public double getScore()
    {
        return score;    }

    public void setScore(double score)
    {
        this.score = score;    }

    public String getStudentId()
    {
        return studentId;    }

    public void setStudentId(String studentId)
    {
        this.studentId = studentId;    }

    /** * Converts numeric score to its standardized letter grade
equivalent.
    * @return Letter grade String (A, B, C, D, or
F) */ public String getLetterGrade() {
    if (score >= 90.0) return "A";        if (score >=
80.0) return "B";        if (score >= 70.0) return
"C";        if (score >= 60.0) return "D";
    return "F";    }

    @Override    public String toString() {        return
String.format("Subject: %-15s | Score: %6.2f | Grade: %s",
subject, score, getLetterGrade());    }
}

```

File 2: Student.java

Java

```

package com.vityarthi.gradebook.model;

import java.io.Serializable;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/** * Domain entity representing a student profile containing academic
records. */ public class Student implements Serializable {    private
static final long serialVersionUID = 1L;

    private String studentId;
    private String name;

```

```

private String email;
private List<Grade> grades;

    public Student(String studentId, String name, String email)
    {
        this.studentId = studentId;        this.name = name;
        this.email = email;                this.grades = new ArrayList<>();
    }

    public String getStudentId()
    {
        return studentId;    }

    public String getName()
    {
        return name;    }

    public void setName(String name)
    {
        this.name = name;    }

    public String getEmail()
    {
        return email;    }

    public void setEmail(String email) {
        this.email = email;
    }

    public List<Grade> getGrades() {        return
        Collections.unmodifiableList(grades);    }

    public void addGrade(Grade grade)
    {
        if (grade != null) {
            this.grades.add(grade);
        }
    }

    /**      * Calculates the mean average of all recorded scores for this
    student profile.
    * @return Average score as double, or 0.0 if no grades are
    recorded.      */    public double calculateAverage() {        if
    (grades.isEmpty()) {            return 0.0;        }        double
    sum = 0.0;        for (Grade g : grades) {            sum +=
    g.getScore();        }        return sum / grades.size();    }

    @Override    public String toString() {        return String.format("ID: %-10s |
    Name: %-20s | Email: %-25s | Total Subjects: %2d",
    studentId, name, email,
    grades.size());    }
}

```

6.2 Utility Processing Components

File 3: Validator.java

Java

```

package com.vityarthi.gradebook.util;

import java.util.regex.Pattern;

/** * Utility class providing static verification methods for domain
inputs. */ public class Validator {

    private static final Pattern EMAIL_PATTERN =        Pattern.compile("^([A-Za-z0-9+_.-]+@[A-Za-
z0-9.-]+|\\.[A-Za-z]{2,}$");

    /**      * Audits student ID validity.      */    public static boolean
    isValidStudentId(String id) {        return id != null &&
    !id.trim().isEmpty() && id.trim().length() <= 20;    }

    /**      * Audits legal name string length.      */
    public static boolean isValidName(String name) {
        return name != null && name.trim().length() >= 2;    }

    /**      * Verifies syntax of email input using regular expression
    matching.      */    public static boolean isValidEmail(String email)
    {
        if (email == null) return false;        return
        EMAIL_PATTERN.matcher(email.trim()).matches();    }

    /**      * Confirms numeric mark falls within acceptable bounds [0.0
    - 100.0].      */    public static boolean isValidScore(double score) {
        return score >= 0.0 && score <= 100.0;    }

    /**      * Verifies that subject name is
    non-empty.
    */
}

```

```

    public static boolean isValidSubject(String subject) {
return subject != null && !subject.trim().isEmpty();    }
}

```

File 4: FileHandler.java

Java

```

package com.vityarthi.gradebook.util;

import com.vityarthi.gradebook.model.Grade;
import com.vityarthi.gradebook.model.Student;

import java.io.*; import
java.util.HashMap;
import java.util.Map;

/** * Utility handling disk persistence using structured flat-file reader/writer
streams. */ public class FileHandler {

    private static final String DATA_DIR = "data";    private static final String
FILE_PATH = DATA_DIR + File.separator + "students.txt";

    /**      * Serializes all current student entities and grades to
disk file.
    * @param studentMap Map of active student records indexed by ID
    */
    public static void saveToFile(Map<String, Student> studentMap)
    {
        File dir = new File(DATA_DIR);        if (!dir.exists())
        {
            dir.mkdirs();        }

        try (BufferedWriter writer = new BufferedWriter(new FileWriter(FILE_PATH)))
        {
            for (Student student : studentMap.values()) {
writer.write("STUDENT:" + student.getStudentId() + "," +
student.getName() + "," + student.getEmail());
writer.newLine();

                for (Grade grade : student.getGrades()) {
writer.write("GRADE:" + grade.getSubject() + "," + grade.getScore());
writer.newLine();                }
writer.write("END");
writer.newLine();
                System.out.println("[SUCCESS] System state saved successfully to '" + FILE_PATH + "'.");
            } catch (IOException e) {
                System.err.println("[ERROR] Failed to save data to disk: " + e.getMessage());
            }
        }

        /**      * Deserializes and restores student data from local disk
storage.
    * @return HashMap containing restored Student objects indexed by Student ID
    */
    public static Map<String, Student> loadFromFile() {
Map<String, Student> studentMap = new HashMap<>();
File file = new File(FILE_PATH);

        if (!file.exists()) {            System.out.println("[INFO] Persistent data file not
found. Initializing empty dataset.");            return studentMap;        }

        try (BufferedReader reader = new BufferedReader(new FileReader(file)))
        {
            String line;            Student currentStudent = null;

            while ((line = reader.readLine()) != null)
            {
                line = line.trim();
                if (line.isEmpty()) continue;

                if (line.startsWith("STUDENT:")) {
String[] parts = line.substring(8).split(",");
if (parts.length == 3) {            String id =
parts[0];
                String name = parts[1];
String email = parts[2];            currentStudent =
new Student(id, name, email);
studentMap.put(id, currentStudent);
                }
            } else if (line.startsWith("GRADE:") && currentStudent != null)
            {
                String[] parts = line.substring(6).split(",");
if (parts.length == 2) {            String subject = parts[0];
                double score = Double.parseDouble(parts[1]);
Grade grade = new Grade(subject, score, currentStudent.getStudentId());
currentStudent.addGrade(grade);            } else if
(line.equals("END")) {            currentStudent = null;        }
            }
            System.out.println("[SUCCESS] Successfully loaded " + studentMap.size() + "
student records from disk.");        } catch (IOException | NumberFormatException e) {

```

```

System.err.println("[ERROR] Error reading data file. Partial data recovered. Cause: " + e.
getMessage());
    }

    return studentMap;
}
}

```

6.3 Service Application Layer

File 5: StudentService.java

Java

```

package com.vityarthi.gradebook.service;

import com.vityarthi.gradebook.model.Student;
import com.vityarthi.gradebook.util.FileHandler;

import java.util.Collection;
import java.util.HashMap;
import java.util.Map;

/** * Service managing profile registration, retrieval, and state
operations. */ public class StudentService {

    private Map<String, Student> studentMap;

    public StudentService() {
        this.studentMap = new HashMap<>();
    }

    public boolean addStudent(Student student) {
        if (student == null
|| studentMap.containsKey(student.getStudentId())) {
            return false;
        }
        studentMap.put(student.getStudentId(), student);
        return true;
    }

    public Student getStudent(String studentId) {
        return studentMap.get(studentId);
    }

    public Collection<Student> getAllStudents() {
        return studentMap.values();
    }

    public boolean studentExists(String studentId) {
        return studentMap.containsKey(studentId);
    }

    public int getTotalStudentCount() {
        return studentMap.size();
    }

    public void persistData() {
        FileHandler.saveToFile(studentMap);
    }

    public void loadData() {
        this.studentMap
= FileHandler.loadFromFile();
    }
}

```

File 6: GradeService.java

Java

```

package com.vityarthi.gradebook.service;

import com.vityarthi.gradebook.model.Grade;
import com.vityarthi.gradebook.model.Student;

/** * Service orchestrating score assignments to student
profiles. */ public class GradeService {

    private StudentService studentService;

    public GradeService(StudentService studentService) {
        this.studentService = studentService;
    }

    /** * Binds a new subject mark entry to an existing student
profile. */
    public boolean addGradeToStudent(String studentId, String subject, double score) {
        Student student = studentService.getStudent(studentId);
        if (student == null) {
            return false;
        }
    }
}

```

```

        Grade grade = new Grade(subject, score,
studentId);        student.addGrade(grade);
return true;    }
}

```

File 7: ReportService.java

Java

```

package com.vityarthi.gradebook.service;

import com.vityarthi.gradebook.model.Grade;
import com.vityarthi.gradebook.model.Student;

import java.util.Collection;

/** * Service compiling transcript output and class
statistics. */ public class ReportService {

    private StudentService studentService;

    public ReportService(StudentService studentService)
    {        this.studentService = studentService;    }

    /**      * Prints a formatted transcript for a single
student.      */
    public void printStudentReport(String studentId) {        Student student =
studentService.getStudent(studentId);        if (student == null) {
System.out.println("[ERROR] Student record not found for ID: " + studentId);
return;    }

        System.out.println("\n=====");
        System.out.println("                INDIVIDUAL STUDENT REPORT                ");
        System.out.println("=====");
        System.out.println(" Student ID      : " + student.getId());
        System.out.println(" Student Name   : " + student.getName());
        System.out.println(" Email Address : " + student.getEmail());
        System.out.println("-----");
        System.out.printf(" %-20s | %-12s | %-10s\n", "SUBJECT", "SCORE", "LETTER GRADE");
System.out.println("-----");

        if (student.getGrades().isEmpty()) {            System.out.println("
[No Grades Recorded for Student]");        } else {
            for (Grade g : student.getGrades()) {
System.out.printf(" %-20s | %-12.2f | %-10s\n",
                    g.getSubject(), g.getScore(), g.getLetterGrade());
            }
            System.out.println("-----");
System.out.printf(" CUMULATIVE AVERAGE : %36.2f%\n", student.calculateAverage());
        }
        System.out.println("=====\n");
    }

    /**      * Aggregates and prints statistical metrics for all registered
students.      */
    public void printClassStatistics() {        Collection<Student>
students = studentService.getAllStudents();

        if (students.isEmpty()) {            System.out.println("[INFO] No
students registered in the system.");            return;        }

        int totalStudents = students.size();
        int evaluatedStudentsCount = 0;
        double sumOfAverages = 0.0;

        Student topStudent = null;
        Student lowestStudent = null;
        double highestAvg = -1.0;
        double lowestAvg = 101.0;

        for (Student s : students) {
            if (!s.getGrades().isEmpty()) {
                evaluatedStudentsCount++;                double
                avg = s.calculateAverage();
                sumOfAverages += avg;

                if (avg > highestAvg)
                {
                    highestAvg =
                    avg;
                    topStudent =
                    s;
                }
                if (avg < lowestAvg) {
                    lowestAvg = avg;
                    lowestStudent = s;                }
            }
        }
    }
}

```

```

    }

    System.out.println("\n=====");
    System.out.println("                      CLASS STATISTICAL REPORT                      ");
    System.out.println("=====");
    System.out.println(" Total Registered Students : " + totalStudents);
    System.out.println(" Students with Grades      : " + evaluatedStudentsCount);

    if (evaluatedStudentsCount == 0) {
        System.out.println(" Status
: No grade data available for stats.");
    } else {
        double classMean = sumOfAverages / evaluatedStudentsCount;
        System.out.printf(" Class Overall Mean Average: %.2f%%\n", classMean);
        System.out.println("-----");
    }

    System.out.printf(" Top Performer      : %s (ID: %s) with Average:
%.2f%%\n",
        topStudent.getName(), topStudent.getStudentId(), highestAvg);
    System.out.printf(" Lowest Performer      : %s (ID: %s) with Average: %.2f%%\n",
        lowestStudent.getName(), lowestStudent.getStudentId(), lowestAvg);
    System.out.println("=====\\n");
}
}

```

6.4 Interactive User Interface Layer

File 8: Main.java

Java

```

package com.vityarthi.gradebook;

import com.vityarthi.gradebook.model.Student; import
com.vityarthi.gradebook.service.GradeService; import
com.vityarthi.gradebook.service.ReportService; import
com.vityarthi.gradebook.service.StudentService;
import com.vityarthi.gradebook.util.Validator;

import java.util.Collection;
import java.util.Scanner;

/** * Main application entry point providing terminal-based control
menus. */ public class Main {

    private static StudentService studentService = new StudentService();
    private static GradeService gradeService = new GradeService(studentService);
    private static ReportService reportService = new ReportService(studentService);
    private static Scanner scanner = new Scanner(System.in);

    public static void main(String[] args) {
        System.out.println("=====");
        System.out.println("          INITIALIZING STUDENT GRADE MANAGEMENT SYSTEM...          ");
        System.out.println("=====");
        // Load persistent data from
        storage file      studentService.loadData();

        boolean running = true;      while
        (running) {      displayMenu();
        System.out.print("Enter choice (1-6): ");
        String input =
        scanner.nextLine().trim();      if
        (input.isEmpty()) continue;

        int choice = -1;      try {      choice = Integer.parseInt(input);
        } catch (NumberFormatException e) {      System.out.println("[ERROR] Invalid input.
Please enter a number between 1 and 6.");      continue;      }

        switch (choice) {      case 1:      handleAddStudent();
        break;      case 2:      handleViewStudents();
        break;      case 3:      handleAddGrade();      break;
        case 4:      handleGenerateReport();      break;
        case 5:      handleViewStatistics();      break;
        case 6:      System.out.println("\n[INFO] Persisting state to disk and closing
system...");      studentService.persistData();      running =
false;      break;      default:
        System.out.println("[ERROR] Choice out of bounds. Select options 1 through 6.");
        }

        System.out.println("=====");
        System.out.println("          SYSTEM TERMINATED SUCCESSFULLY. HAVE A GREAT DAY!          ");
        System.out.println("=====");
        scanner.close();
    }

    private static void displayMenu() {      System.out.println("\n-----
-----");
    }
}

```

```

        System.out.println("                                MAIN CONTROL MENU                                ");
        System.out.println("-----");
        System.out.println(" 1. Add New Student Profile");
        System.out.println(" 2. View All Registered Students");
        System.out.println(" 3. Record Grade for Student");
        System.out.println(" 4. Generate Individual Student Report");
        System.out.println(" 5. View Class-wide Statistics");
        System.out.println(" 6. Save Data & Exit System");
        System.out.println("-----");
    }

    private static void handleAddStudent() {
        System.out.println("\n--- [MODULE: ADD NEW STUDENT] ---");

        System.out.print("Enter Student ID: ");
        String id = scanner.nextLine().trim();
        if (!Validator.isValidStudentId(id)) {
            System.out.println("[ERROR] Invalid ID format. Must be non-empty and <= 20 characters.");
            return;
        }

        if (studentService.studentExists(id)) {
            System.out.println("[ERROR] Student profile with ID '" + id + "' already exists.");
            return;
        }

        System.out.print("Enter Student Full Name: ");
        String name = scanner.nextLine().trim();
        if (!Validator.isValidName(name)) {
            System.out.println("[ERROR] Invalid name length. Must be at least 2 characters.");
            return;
        }

        System.out.print("Enter Student Email: ");
        String email = scanner.nextLine().trim();
        if (!Validator.isValidEmail(email)) {
            System.out.println("[ERROR] Invalid email format (e.g., user@domain.com).");
            return;
        }

        Student student = new Student(id, name, email);
        if (studentService.addStudent(student)) {
            System.out.println("[SUCCESS] Student profile created for '" + name + "' (ID: " + id + ").");
        } else {
            System.out.println("[ERROR] Failed to save student record.");
        }
    }

    private static void handleViewStudents() {
        System.out.println("\n--- [MODULE: REGISTERED STUDENTS LIST] ---");
        Collection<Student> students = studentService.getAllStudents();

        if (students.isEmpty()) {
            System.out.println("[INFO] Zero student records stored in system.");
            return;
        }

        System.out.println("-----");
        for (Student s : students) {
            System.out.println(s);
        }
        System.out.println("-----");
        System.out.println("Total Count: " + students.size() + " student(s).");
    }

    private static void handleAddGrade() {
        System.out.println("\n--- [MODULE: RECORD SUBJECT GRADE] ---");
        System.out.print("Enter Target Student ID: ");
        String id = scanner.nextLine().trim();

        if (!studentService.studentExists(id)) {
            System.out.println("[ERROR] No student profile found for ID '" + id + "'.");
            return;
        }

        System.out.print("Enter Subject Name (e.g., Mathematics, Physics): ");
        String subject = scanner.nextLine().trim();
        if (!Validator.isValidSubject(subject)) {
            System.out.println("[ERROR] Subject name cannot be empty.");
            return;
        }

        System.out.print("Enter Numeric Score (0.0 - 100.0): ");
        String scoreInput = scanner.nextLine().trim();
        double score;
        try {
            score = Double.parseDouble(scoreInput);
        } catch (NumberFormatException e) {
            System.out.println("[ERROR] Score must be a valid decimal number.");
            return;
        }

        if (!Validator.isValidScore(score)) {
            System.out.println("[ERROR] Score must fall strictly within 0.0 - 100.0 range.");
            return;
        }

        if (gradeService.addGradeToStudent(id, subject, score)) {
            System.out.println("[SUCCESS] Grade recorded successfully for Subject: " + subject);
        } else {
            System.out.println("[ERROR] Grade assignment failed.");
        }
    }
}

```

```

        private static void handleGenerateReport() {
            System.out.println("\n--- [MODULE: GENERATE TRANSCRIPT] ---");
            System.out.print("Enter Target Student ID: ");
            String id = scanner.nextLine().trim();
            reportService.printStudentReport(id);
        }
        private static void handleViewStatistics()
        {
            reportService.printClassStatistics();
        }
    }
}

```

7. SYSTEM EXECUTION LOGS AND USER INTERFACE DEMONSTRATION

7.1 Startup and Main Interface Launch

Plaintext

```

=====
INITIALIZING STUDENT GRADE MANAGEMENT SYSTEM...
=====
[SUCCESS] Successfully loaded 2 student records from disk.

-----
                        MAIN CONTROL MENU
-----
1. Add New Student Profile
2. View All Registered Students
3. Record Grade for Student
4. Generate Individual Student Report
5. View Class-wide Statistics
6. Save Data & Exit System
-----Enter
choice (1-6):

```

7.2 Student Registration Interaction

Plaintext

```

Enter choice (1-6): 1

--- [MODULE: ADD NEW STUDENT] ---
Enter Student ID: STU-8092
Enter Student Full Name: Alex Mercer
Enter Student Email: alex.mercer@university.edu [SUCCESS] Student
profile created for 'Alex Mercer' (ID: STU-8092).

```

7.3 Grade Logging Interaction

Plaintext

```

Enter choice (1-6): 3

--- [MODULE: RECORD SUBJECT GRADE] ---
Enter Target Student ID: STU-8092
Enter Subject Name (e.g., Mathematics, Physics): Computer Architecture
Enter Numeric Score (0.0 - 100.0): 94.5
[SUCCESS] Grade recorded successfully for Subject: Computer Architecture

```

7.4 Individual Transcript Generation Output

Plaintext

```

Enter choice (1-6): 4

--- [MODULE: GENERATE TRANSCRIPT] ---

```

```
Enter Target Student ID: STU-8092

=====
                        INDIVIDUAL STUDENT REPORT
=====
Student ID      : STU-8092
Student Name    : Alex Mercer
Email Address   : alex.mercer@university.edu
=====
SUBJECT          | SCORE          | LETTER GRADE
=====
Computer Architecture|      94.50 | A
Calculus II       |      88.00 | B
Linear Algebra    |      91.50 | A
=====
CUMULATIVE AVERAGE :                      91.33%
=====
```

7.5 Global Analytics Output

Plaintext

```
Enter choice (1-6): 5

=====
                        CLASS STATISTICAL REPORT
=====
Total Registered Students : 3
Students with Grades      : 3
Class Overall Mean Average: 87.42%
=====
Top Performer              : Alex Mercer (ID: STU-8092) with Average: 91.33%
Lowest Performer           : John Doe (ID: STU-1001) with Average: 82.50%
=====
```

8. SYSTEM VERIFICATION, TESTING AND ANALYSIS

```
=====
                        TESTING SUITE SUMMARY
=====
[UNIT TESTING]           [INTEGRATION TESTING]       [SYSTEM PERSISTENCE]
+-- Input Rules Validation +-- Profile-Grade Linkage +-- Cold Re-boot Test
+-- Score Range Boundaries +-- Average Calculations  +-- File Corruption Check
=====
```

8.1 Verification Test Executions

- **Test Case 01 - Student Registration:** Valid inputs (ID: 101, Name: David, Email: d@test.com). Verified successful profile creation and inclusion in student directory.
- **Test Case 02 - Duplicate ID Protection:** Attempted re-registration of ID 101. System triggered validation block and output duplicate error notice.
- **Test Case 03 - Email Validation Rule:** Provided malformed input david.test.com missing the @ symbol. System caught syntax error and prompted for retry.
- **Test Case 04 - Score Boundary Verification:** Entered numeric score 105.0. System rejected entry as out-of-bounds (\$> 100.0\$).
- **Test Case 05 - Lower Score Boundary Verification:** Entered numeric score -5.0. System rejected entry as out-of-bounds (\$< 0.0\$).
- **Test Case 06 - Letter Grade Mapping Verification:** Tested score 89.9 (mapped to B) and score 90.0 (mapped to A). Verified correct letter grade assignment logic.
- **Test Case 07 - Cumulative Average Calculation:** Added scores 80.0 and 90.0. System correctly calculated average as 85.0%.

- **Test Case 08 - Persistence State Recovery:** Terminated application with active records, then relaunched. Verified 100% data restoration from `data/students.txt`.

9. TECHNICAL CHALLENGES AND RESOLUTIONS

9.1 Scanner Newline Character Residuals

- **Issue:** Mixed calls to standard scanner numeric methods (like `nextInt()`) leave trailing newline characters in the buffer, causing subsequent `nextLine()` prompts to read blank entries automatically.
- **Resolution:** Standardized input processing by reading all user entries as Strings using `scanner.nextLine()`, then parsing them into numeric types using explicit static converter methods (`Integer.parseInt()`, `Double.parseDouble()`).

9.2 Flat-File Parsing and Fault Tolerance

- **Issue:** External text file edits or malformed lines can crash standard text stream parsers during application initialization.
- **Resolution:** Added clear line marker tokens (`STUDENT:`, `GRADE:`, `END`) to delineate records. Implemented localized try-catch blocks inside `FileHandler.java` to skip corrupted lines safely while preserving valid data entries.

9.3 Boundary Edge Cases in Statistical Calculations

- **Issue:** Evaluating minimum averages on empty class lists or student profiles without grades caused divide-by-zero errors or incorrect lowest score assignments.
- **Resolution:** Added structural state checks to verify that student grade lists are non-empty before processing statistical calculations, setting sensible default initializers for high and low score markers.

10. CONCLUSION AND FUTURE ENHANCEMENTS

10.1 Conclusion

The Student Grade Management System fulfills all core functional requirements for academic record management, score entry, letter grade assignment, and class performance reporting. By utilizing a modular, multi-tier software architecture built on Java Standard Edition, the application guarantees maintainability, high execution stability, constant-time data lookups, and total storage persistence without relying on third-party frameworks.

10.2 Future Scope

- **Database Driver Integration:** Upgrading the storage layer from flat files to a Relational Database Management System (such as PostgreSQL or MySQL) using JDBC.
- **Graphical User Interface (GUI):** Developing a modern visual interface using JavaFX or Swing to complement the existing terminal menu structure.
- **Role-Based Access Control (RBAC):** Implementing multi-user authentication tiers to separate Administrator privileges from Student view-only access.
- **Exporting Options:** Adding automated export utilities to render transcripts directly as formatted PDF or CSV documents.

=====

END OF ACADEMIC REPORT

=====